

UNIVERSIDAD TECNOLÓGICA NACIONAL

Facultad Regional General Pacheco

Tecnicatura Universitaria en Programación

Materia: Base de Datos II

TRABAJO PRÁCTICO INTEGRADOR

**Sistema de Gestión de Gimnasio y Club de Alto Rendimiento
(GimnasioDB)**

Grupo	17
Integrantes	Juan Cruz Dominguez Pistoia - Legajo: 32444 Lautaro Silva – Legajo: 32647

1. EXPLICACIÓN DEL SISTEMA

El sistema consiste en una solución de software empresarial diseñada para la administración integral, control operativo y auditoría financiera de un gimnasio o complejo deportivo de alto rendimiento. El núcleo de la base de datos se encarga de centralizar y procesar toda la información de flujos logísticos, comerciales y de recursos humanos del establecimiento, resolviendo problemáticas comunes asociadas a la redundancia de datos y a la falta de consistencia en el control de accesos, cobros y cupos de actividades.

Estructuralmente, el diseño arquitectónico de la base de datos implementa el concepto de generalización/especialización (herencia) a través de una entidad mímica principal denominada Personas. A partir de ella se desprenden las entidades especializadas Socios y Empleados

mediante relaciones uno a muchos flexibles. Esta decisión técnica de modelado responde directamente a la necesidad de soportar de manera escalable y limpia una condición típica del negocio: que un mismo individuo pueda formar parte de la nómina de empleados del gimnasio (por ejemplo, un profesor de planta o administrativo) y, en simultáneo, estar registrado como un socio activo que utiliza las instalaciones recreativas o de entrenamiento, evitando por completo la duplicación de datos de contacto o de identidad.

2. FUNCIONALIDADES PRINCIPALES

El sistema divide su alcance operativo en dos grandes módulos funcionales parametrizados según el rol del usuario que interactúe con la plataforma informática:

2.1. Módulo Operativo para Socios

- **Gestión de Identidad:** Permite el alta, modificación y mantenimiento autónomo de los datos personales (Nombres, Apellidos, Teléfono, Correo Electrónico Único y Fecha de Nacimiento).
- **Contratación y Control de Membresías:** Posibilita la adquisición, renovación o actualización de los pases comerciales vigentes del club (Pase Libre Mensual, Pase por Días, Pase Anual), permitiendo al socio consultar de manera instantánea el estado de vigencia, la fecha de inicio, la fecha de vencimiento y el indicador lógico de habilitación de acceso.
- **Inscripción Dinámica a Clases:** Facilita la visualización del cronograma logístico de las actividades dictadas en el gimnasio, permitiendo al socio inscribirse de forma presencial o remota en una clase específica, interactuando de manera directa con validaciones del motor de base de datos que controlan la disponibilidad de vacantes en tiempo real.
- **Historial de Presentismo:** Registra las asistencias diarias a las ranuras de entrenamiento programadas para que el socio audite su nivel de presentismo y constancia mensual.

2.2. Módulo de Administración y Recursos Humanos

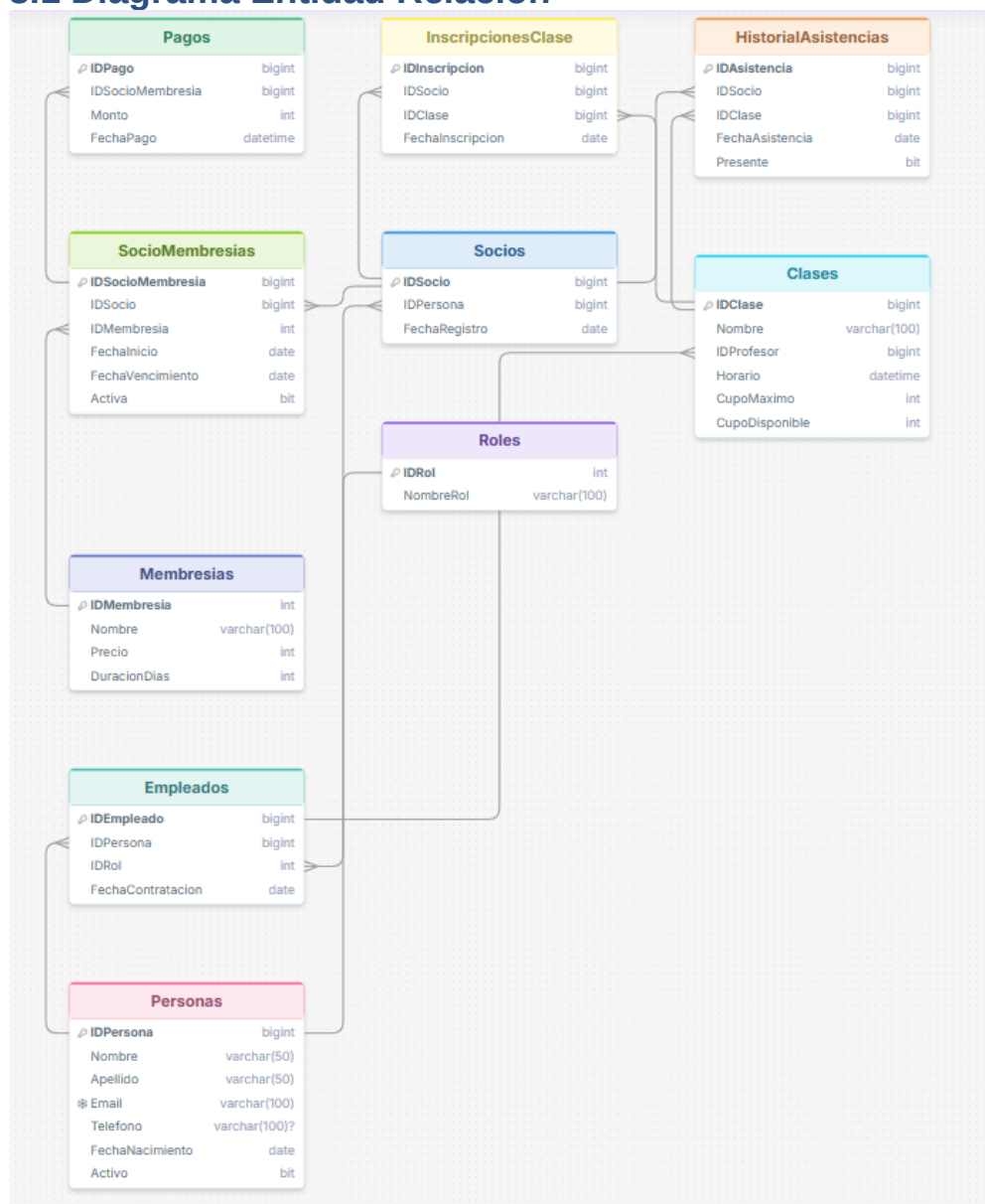
- **Administración de Personal y Roles:** Centraliza el control de todo el staff activo del establecimiento (profesores de planta, personal trainers, recepcionistas, administradores y personal de limpieza) mediante la asignación estricta de roles, permitiendo escalar la estructura organizativa sin modificar el esquema relacional.
- **Planificación Logística de Actividades:** Permite dar de alta el cronograma de clases semanales del complejo, asignando de forma unívoca al empleado responsable del dictado, el horario exacto, la denominación de la actividad y la capacidad máxima del salón (cupó límite).
- **Procesamiento y Cobro de Cuotas:** Procesa las transacciones de caja, permitiendo el

ingreso de pagos totales o parciales (cuotas) enlazados unívocamente a un contrato de membresía socio-establecimiento, manejando alta precisión en el resguardo de dinero mediante el tipo de dato nativo MONEY.

- **Auditoría y Reportes de Gestión:** Provee a la gerencia de tableros de información consolidados mediante vistas complejas para controlar la facturación mensual, el índice de ocupación de las salas de entrenamiento y el listado automatizado de pases morosos o vencidos.

3. ARQUITECTURA DE LA BASE DE DATOS

3.1 Diagrama Entidad Relación



3.2 Script Creación de la Base de Datos

A continuación, se detalla el código Transact-SQL estructurado de manera modular y jerárquica para la inicialización y despliegue del esquema de datos relacional de la base GimnasioDB sobre un motor Microsoft SQL Server:

```
CREATE DATABASE GimnasioDB;
GO
USE GimnasioDB;
GO

CREATE TABLE Personas (
    IDPersona BIGINT IDENTITY(1,1) NOT NULL,
    Nombre VARCHAR(50) NOT NULL,
    Apellido VARCHAR(50) NOT NULL,
    Email VARCHAR(100) NOT NULL,
    Telefono VARCHAR(100) NULL,
    FechaNacimiento DATE NOT NULL,
    Activo BIT NOT NULL,

    CONSTRAINT PK_Personas PRIMARY KEY (IDPersona),
    CONSTRAINT UQ_Personas_Email UNIQUE (Email)
);
GO

CREATE TABLE Roles (
    IDRol INT IDENTITY(1,1) NOT NULL,
    NombreRol VARCHAR(100) NOT NULL,

    CONSTRAINT PK_Roles PRIMARY KEY (IDRol)
);
GO

CREATE TABLE Membresias (
    IDMembresia INT IDENTITY(1,1) NOT NULL,
    Nombre VARCHAR(100) NOT NULL,
    Precio MONEY NOT NULL,
    DuracionDias INTEGER NOT NULL,

    CONSTRAINT PK_Membresias PRIMARY KEY (IDMembresia)
);
GO

CREATE TABLE Socios (
    IDSocio BIGINT IDENTITY(1,1) NOT NULL,
    IDPersona BIGINT NOT NULL,
    FechaRegistro DATE NOT NULL,

    CONSTRAINT PK_Socios PRIMARY KEY (IDSocio),
```

```

        CONSTRAINT FK_Socios_Personas FOREIGN KEY (IDPersona) REFERENCES
Personas (IDPersona)
);
GO

CREATE TABLE Empleados (
    IDEmpleado BIGINT IDENTITY(1,1) NOT NULL,
    IDPersona BIGINT NOT NULL,
    IDRol INT NOT NULL,
    FechaContratacion DATE NOT NULL,

    CONSTRAINT PK_Empleados PRIMARY KEY (IDEmpleado),
    CONSTRAINT FK_Empleados_Personas FOREIGN KEY (IDPersona) REFERENCES
Personas (IDPersona),
    CONSTRAINT FK_Empleados_Roles FOREIGN KEY (IDRol) REFERENCES Roles (IDRol)
);
GO

CREATE TABLE Clases (
    IDClase BIGINT IDENTITY(1,1) NOT NULL,
    Nombre VARCHAR(100) NOT NULL,
    IDProfesor BIGINT NOT NULL,
    Horario DATETIME NOT NULL,
    CupoMaximo INTEGER NOT NULL,
    CupoDisponible INTEGER NOT NULL,

    CONSTRAINT PK_Clases PRIMARY KEY (IDClase),
    CONSTRAINT FK_Clases_Empleados FOREIGN KEY (IDProfesor) REFERENCES
Empleados (IDEmpleado)
);
GO

CREATE TABLE SocioMembresias (
    IDSocioMembresia BIGINT IDENTITY(1,1) NOT NULL,
    IDSocio BIGINT NOT NULL,
    IDMembresia INT NOT NULL,
    FechaInicio DATE NOT NULL,
    FechaVencimiento DATE NOT NULL,
    Activa BIT NOT NULL,

    CONSTRAINT PK_SocioMembresias PRIMARY KEY (IDSocioMembresia),
    CONSTRAINT FK_SocioMembresias_Socios FOREIGN KEY (IDSocio) REFERENCES
Socios (IDSocio),
    CONSTRAINT FK_SocioMembresias_Membresias FOREIGN KEY (IDMembresia) REFERENCES
Membresias (IDMembresia),
    CONSTRAINT UQ_SocioMembresia_Activa UNIQUE (IDSocio, IDMembresia, FechaInicio)
);
GO

CREATE TABLE Pagos (
    IDPago BIGINT IDENTITY(1,1) NOT NULL,
    IDSocioMembresia BIGINT NOT NULL,
    Monto MONEY NOT NULL,

```

```

    FechaPago DATETIME NOT NULL,

    CONSTRAINT PK_Pagos PRIMARY KEY (IDPago),
    CONSTRAINT FK_Pagos_SocioMembresias FOREIGN KEY (IDSocioMembresia) REFERENCES
SocioMembresias (IDSocioMembresia)
);
GO

CREATE TABLE InscripcionesClase (
    IDInscripcion BIGINT IDENTITY(1,1) NOT NULL,
    IDSocio BIGINT NOT NULL,
    IDClase BIGINT NOT NULL,
    FechaInscripcion DATE NOT NULL,

    CONSTRAINT PK_InscripcionesClase PRIMARY KEY (IDInscripcion),
    CONSTRAINT FK_InscripcionesClase_Socios FOREIGN KEY (IDSocio) REFERENCES
Socios (IDSocio),
    CONSTRAINT FK_InscripcionesClase_Clases FOREIGN KEY (IDClase) REFERENCES
Clases (IDClase)
);
GO

CREATE TABLE HistorialAsistencias (
    IDAsistencia BIGINT IDENTITY(1,1) NOT NULL,
    IDSocio BIGINT NOT NULL,
    IDClase BIGINT NOT NULL,
    FechaAsistencia DATE NOT NULL,
    Presente BIT NOT NULL,

    CONSTRAINT PK_HistorialAsistencias PRIMARY KEY (IDAsistencia),
    CONSTRAINT FK_HistorialAsistencias_Socios FOREIGN KEY (IDSocio) REFERENCES
Socios (IDSocio),
    CONSTRAINT FK_HistorialAsistencias_Clases FOREIGN KEY (IDClase) REFERENCES
Clases (IDClase)
);
GO

```

4. IMPLEMENTACIÓN DE OBJETOS CLAVE EN EL SISTEMA

4.1. Vistas

Las vistas implementadas encapsulan uniones complejas y funciones agregadas para facilitar la lectura inmediata de datos críticos de gestión por parte del personal de auditoría o de administración, sin exponer el diseño físico de las tablas base:

```

-- VISTA 1: Monitoreo de vigencia de contratos de membresia activos
CREATE VIEW vw_SociosMembresiasActivas AS
SELECT
    s.IDSocio,
    p.Nombre + ' ' + p.Apellido AS Socio,
    p.Email,
    m.Nombre AS PlanContratado,
    sm.FechaInicio,
    sm.FechaVencimiento,
    DATEDIFF(day, GETDATE(), sm.FechaVencimiento) AS DiasRestantes
FROM SocioMembresias sm
INNER JOIN Socios s ON sm.IDSocio = s.IDSocio
INNER JOIN Personas p ON s.IDPersona = p.IDPersona
INNER JOIN Membresias m ON sm.IDMembresia = m.IDMembresia
WHERE sm.Activa = 1;
GO

-- VISTA 2: Control analítico de ocupación y calculo porcentual de vacantes
CREATE VIEW vw_OcupacionClases AS
SELECT
    c.IDClase,
    c.Nombre AS Clase,
    c.Horario,
    prof_p.Nombre + ' ' + prof_p.Apellido AS Profesor,
    c.CupoMaximo,
    c.CupoDisponible,
    (c.CupoMaximo - c.CupoDisponible) AS AlumnosInscritos,
    CONVERT(DECIMAL(5,2), ((c.CupoMaximo - c.CupoDisponible) * 100.0 /
    c.CupoMaximo)) AS PorcentajeOcupacion
FROM Clases c
INNER JOIN Empleados e ON c.IDProfesor = e.IDEmpleado
INNER JOIN Personas prof_p ON e.IDPersona = prof_p.IDPersona;
GO

-- VISTA 3: Reporte financiero de caja consolidado por periodo mensual
CREATE VIEW vw_RecaudacionMensual AS
SELECT
    YEAR(pa.FechaPago) AS Anio,
    MONTH(pa.FechaPago) AS Mes,
    COUNT(pa.IDPago) AS CantidadTransacciones,
    SUM(pa.Monto) AS TotalRecaudado,
    AVG(pa.Monto) AS TicketPromedio
FROM Pagos pa
GROUP BY YEAR(pa.FechaPago), MONTH(pa.FechaPago);
GO

```

4.2. Procedimientos Almacenados

Siguiendo la consigna de la cátedra, se desarrollaron dos procedimientos almacenados: uno con

lógica exclusiva de lectura estructurada mediante parámetros definidos, y otro orientado a acciones de modificación que utiliza control estricto de transacciones atómicas (BEGIN TRANSACTION / COMMIT / ROLLBACK) y bloques estructurados de captura de excepciones (TRY...CATCH) para asegurar que la caja y los estados lógicos nunca queden en un punto inconsistente.

```
-- PROCEDIMIENTO 1: Reporte parametrizado historico de asistencia por socio
CREATE PROCEDURE sp_ReporteAsistenciasSocio
    @IDSocio BIGINT,
    @FechaDesde DATE,
    @FechaHasta DATE
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        ha.FechaAsistencia,
        c.Nombre AS Clase,
        c.Horario AS HorarioClase,
        CASE WHEN ha.Presente = 1 THEN 'PRESENTE' ELSE 'AUSENTE' END AS
EstadoAsistencia
    FROM HistorialAsistencias ha
    INNER JOIN Clases c ON ha.IDClase = c.IDClase
    WHERE ha.IDSocio = @IDSocio
        AND ha.FechaAsistencia BETWEEN @FechaDesde AND @FechaHasta
    ORDER BY ha.FechaAsistencia DESC;
END;
GO

-- PROCEDIMIENTO 2: Registro seguro y transaccional de pagos de membresias
CREATE PROCEDURE sp_RegistrarPagoMembresia
    @IDSocioMembresia BIGINT,
    @Monto MONEY,
    @FechaPago DATETIME
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        BEGIN TRANSACTION;
        IF NOT EXISTS (SELECT 1 FROM SocioMembresias WHERE IDSocioMembresia =
@IDSocioMembresia)
            BEGIN
                RAISERROR('El ID de contrato Socio-Membresia no existe en el
sistema.', 16, 1);
            END

        INSERT INTO Pagos (IDSocioMembresia, Monto, FechaPago)
        VALUES (@IDSocioMembresia, @Monto, @FechaPago);

        UPDATE SocioMembresias
```



```

        SET Activa = 1
        WHERE IDSocioMembresia = @IDSocioMembresia;

    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0
        ROLLBACK TRANSACTION;

    DECLARE @ErrorMessage NVARCHAR(4000) = ERROR_MESSAGE();
    RAISERROR(@ErrorMessage, 16, 1);
END CATCH
END;
GO

```

4.3. Triggers

Los triggers de la base de datos actúan de forma desatendida a nivel DML, resguardando la lógica logística del gimnasio. El primero se ejecuta tras una inserción (AFTER INSERT) controlando que no se sobrepasen las vacantes del salón, mientras que el segundo reacciona ante cancelaciones o eliminaciones (AFTER DELETE) liberando la disponibilidad del cupo de manera masiva segura.

```

-- TRIGGER 1: Control de cupos automatizado tras insercion de reserva
CREATE TRIGGER trg_Inscripcion_ActualizarCupo
ON InscripcionesClase
AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;

    IF EXISTS (
        SELECT 1
        FROM Clases c
        INNER JOIN inserted i ON c.IDClase = i.IDClase
        WHERE c.CupoDisponible <= 0
    )
    BEGIN
        RAISERROR('Operación cancelada: Cupo totalmente agotado en la clase
seleccionada.', 16, 1);
        ROLLBACK TRANSACTION;
        RETURN;
    END

    UPDATE Clases
    SET CupoDisponible = CupoDisponible - 1
    FROM Clases c
    INNER JOIN inserted i ON c.IDClase = i.IDClase;

```

```

END;
GO

-- TRIGGER 2: Devolucion dinámica de vacantes tras cancelación de reserva
CREATE TRIGGER trg_Cancelacion_ActualizarCupo
ON InscripcionesClase
AFTER DELETE
AS
BEGIN
    SET NOCOUNT ON;

    UPDATE Clases
    SET CupoDisponible = CupoDisponible + 1
    FROM Clases c
    INNER JOIN deleted d ON c.IDClase = d.IDClase;
END;
GO

```

5. ENLACES A RECURSOS COMPLEMENTARIOS

Para evaluar el despliegue del entorno y auditar la suite completa de scripts ejecutados de manera ordenada, se adjuntan las direcciones de control del proyecto:

- **Repositorio del Proyecto en GitHub:**
<https://github.com/Cruplusplus/UTNFRGP-TUP-BD2-TPI-G17>
- **Video Defensa Demostrativo del Funcionamiento:**
<https://drive.google.com/file/d/17f6k1G5ZnylehHprC3YnEzHT0-CnmU0I/view?usp=sharing>